BUILD WITH CLAUDE CODE

Get started with Claude Code hooks

Learn how to customize and extend Claude Code's behavior by registering shell commands

Claude Code hooks are user-defined shell commands that execute at various points in Claude Code's lifecycle. Hooks provide deterministic control over Claude Code's behavior, ensuring certain actions always happen rather than relying on the LLM to choose to run them.

For reference documentation on hooks, see Hooks reference.

Example use cases for hooks include:

- **Notifications**: Customize how you get notified when Claude Code is awaiting your input or permission to run something.
- **Automatic formatting**: Run `prettier` on .ts files, `gofmt` on .go files, etc. after every file edit.
- **Logging**: Track and count all executed commands for compliance or debugging.
- **Feedback**: Provide automated feedback when Claude Code produces code that does not follow your codebase conventions.
- **Custom permissions**: Block modifications to production files or sensitive directories.

By encoding these rules as hooks rather than prompting instructions, you turn suggestions into app-level code that executes every time it is expected to run.

You must consider the security implication of hooks as you add them, because hooks run automatically during the agent loop with your current environment's creden
example, malicious hooks code can exfiltrate your data. Always review your
implementation before registering them.

Ask Docs

Hook Events Overview

Claude Code provides several hook events that run at different points in the workflow:

- **PreToolUse**: Runs before tool calls (can block them)
- **PostToolUse**: Runs after tool calls complete
- **UserPromptSubmit**: Runs when the user submits a prompt, before Claude processes it
- **Notification**: Runs when Claude Code sends notifications
- **Stop**: Runs when Claude Code finishes responding
- **SubagentStop**: Runs when subagent tasks complete
- **PreCompact**: Runs before Claude Code is about to run a compact operation
- **SessionStart**: Runs when Claude Code starts a new session or resumes an existing session
- **SessionEnd**: Runs when Claude Code session ends

Each event receives different data and can control Claude's behavior in different ways.

Quickstart

In this quickstart, you'll add a hook that logs the shell commands that Claude Code runs.

Prerequisites

Install `jq` for JSON processing in the command line.

Step 1: Open hooks configuration

Run the `/hooks` slash command and select the `PreToolUse` hook event.

`PreToolUse` hooks run before tool calls and can block them while providing Claude feedback on what to do differently.

You can use `*` to match all tools.

Step 3: Add the hook

Select `+ Add new hook…` and enter this command:

```
jq -r '"\(.tool_input.command) - \(.tool_input.description // "No description")'
```

Step 4: Save your configuration

For storage location, select `User settings` since you're logging to your home directory. This hook will then apply to all projects, not just your current project.

Then press Esc until you return to the REPL. Your hook is now registered!

Step 5: Verify your hook

Run `/hooks` again or check `~/.claude/settings.json` to see your configuration:

```
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '\"\\(.tool_input.command) - \\(.tool_input.descri
          }
        ]
      }
    ]
  }
}
```

Step 6: Test your hook

Ask Claude to run a simple command like `ls` and check your log file:

```
cat ~/.claude/bash-command-log.txt
```

You should see entries like:

```
ls - Lists files and directories
```

More Examples

For a complete example implementation, see the **bash command validator example** in our public codebase.

Code Formatting Hook

Automatically format TypeScript files after editing:

```
        "matcher": "Edit|MultiEdit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | { read file_path; if ech
          }
        ]
      }
    ]
  }
}
```

Markdown Formatting Hook

Automatically fix missing language tags and formatting issues in markdown files:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|MultiEdit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/markdown_formatter
          }
        ]
      }
    ]
  }
}
```

Create `.claude/hooks/markdown_formatter.py` with this content:

```
"""
import json
import sys
import re
import os

def detect_language(code):
    """Best-effort language detection from code content."""
    s = code.strip()

    # JSON detection
    if re.search(r'^\s*[{\[]', s):
        try:
            json.loads(s)
            return 'json'
        except:
            pass

    # Python detection
    if re.search(r'^\s*def\s+\w+\s*\(', s, re.M) or \
       re.search(r'^\s*(import|from)\s+\w+', s, re.M):
        return 'python'

    # JavaScript detection
    if re.search(r'\b(function\s+\w+\s*\(|const\s+\w+\s*=)', s) or \
       re.search(r'=>|console\.(log|error)', s):
        return 'javascript'

    # Bash detection
    if re.search(r'^#!.*\b(bash|sh)\b', s, re.M) or \
       re.search(r'\b(if|then|fi|for|in|do|done)\b', s):
        return 'bash'

    # SQL detection
    if re.search(r'\b(SELECT|INSERT|UPDATE|DELETE|CREATE)\s+', s, re.I):
        return 'sql'

    return 'text'
```

```
        if not info.strip():
            lang = detect_language(body)
            return f"{indent}```{lang}\n{body}{closing}\n"
        return match.group(0)

    fence_pattern = r'(?ms)^([ \t]{0,3})```([^\n]*)\n(.*?)(\n\1```)\s*$'
    content = re.sub(fence_pattern, add_lang_to_fence, content)

    # Fix excessive blank lines (only outside code fences)
    content = re.sub(r'\n{3,}', '\n\n', content)

    return content.rstrip() + '\n'

# Main execution
try:
    input_data = json.load(sys.stdin)
    file_path = input_data.get('tool_input', {}).get('file_path', '')

    if not file_path.endswith(('.md', '.mdx')):
        sys.exit(0)  # Not a markdown file

    if os.path.exists(file_path):
        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read()

        formatted = format_markdown(content)

        if formatted != content:
            with open(file_path, 'w', encoding='utf-8') as f:
                f.write(formatted)
            print(f"✓ Fixed markdown formatting in {file_path}")

except Exception as e:
    print(f"Error formatting markdown: {e}", file=sys.stderr)
    sys.exit(1)
```

Make the script executable:

- Detects programming languages in unlabeled code blocks
- Adds appropriate language tags for syntax highlighting
- Fixes excessive blank lines while preserving code content
- Only processes markdown files (`.md` , `.mdx`)

Custom Notification Hook

Get desktop notifications when Claude needs input:

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "notify-send 'Claude Code' 'Awaiting your input'"
          }
        ]
      }
    ]
  }
}
```

File Protection Hook

Block edits to sensitive files:

```
        "matcher": "Edit|MultiEdit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "python3 -c \"import json, sys; data=json.load(sys.stdin)
          }
        ]
      }
    ]
  }
}
```

Learn more

- For reference documentation on hooks, see **Hooks reference**.
- For comprehensive security best practices and safety guidelines, see **Security Considerations** in the hooks reference documentation.
- For troubleshooting steps and debugging techniques, see **Debugging** in the hooks reference documentation.

Was this page helpful? Yes No

Output styles Headless mode

Economic Futures
Research
News
Trust center
Transparency

Support center

Learn

Courses
MCP connectors
Customer stories
Engineering blog
Events
Powered by Claude
Service partners
Startups program

Terms and policies

Privacy policy
Disclosure policy
Usage policy
Commercial terms
Consumer terms